Interface from JDK 1.8V

* From JDK 1.8V, in an interface we can write default and static methods.

Limitation of abstract method while working with a project :

The major inconvenience problem with interface is, if we add any new abstract method at the later stage of development inside an **existing interface** then all the implementer classes have to override that abstract method otherwise the implemented class will become as an abstract class as it is one kind of foundation.

We need to provide implementation for all the abstract methods available inside an interface whether it is required or not.

To avoid this maintenance problem java software people introduced default method inside an interface.

What is default method

It is possible to write **default method** (default keyword + method body) inside an interface from JDK 1.8V to avoid the problem **UnImplEXEC**.

A default method we can declare only inside an interface but not in a class.

A default method must be declared with **default keyword** and contains method body.

By default access modifier of a default method is public.

A default method cannot be method as **abstract, final or static**.

A default method may be overridden by a class that implements the interface defined we can write.

It is mainly used to provide "DEFAULT IMPLEMENTATION"

```
package com.rust.default_method;

public interface Vehicle
{
    void run();
    void horn();

    default void digitalMeter() //JDK 1.8V IAM is public()
    {
        IO.println("Providing default implementation");
    }
}

package com.rust.default_method;

public class Bike implements Vehicle
{
    @Override
    public void run()
    {
        IO.println("Bike is running");
    }

    @Override
    public void horn()
    {
        IO.println("Bike has horn");
    }


}


package com.rust.default_method;

public class Car implements Vehicle
{
    @Override
    public void run()
    {
        IO.println("Car is running");
    }

    @Override
    public void horn()
    {
        IO.println("Car has horn");
    }

    @Override
    public void digitalMeter() //JDK 1.8V
    {
        IO.println("Car has digital meter");
    }
}

package com.rust.default_method;

public class DefaultMethodDemo
{
    public static void main(String[] ...)
    {
        Vehicle v = null;
        v = new Bike(); v.run(); v.horn();
        v = new Car(); v.run(); v.horn(); v.digitalMeter();
    }
}
```

Can we achieve multiple inheritance using default method ?